

MC714 - Sistemas Distribuidos

slides by Maarten van Steen
(adapted from Distributed System - 3rd Edition)

Chapter 08: Fault Tolerance

Version: June 14, 2022

Dependability

Basics

A **component** provides **services** to **clients**. To provide services, the component may require the services from other components $\Rightarrow$ a component may **depend** on some other component.

Specifically

A component C depends on C^* if the **correctness** of C 's behavior depends on the correctness of C^* 's behavior. (Components are processes or channels.)

Dependability

Basics

A **component** provides **services** to **clients**. To provide services, the component may require the services from other components $\Rightarrow$ a component may **depend** on some other component.

Specifically

A component C depends on C^* if the **correctness** of C 's behavior depends on the correctness of C^* 's behavior. (Components are processes or channels.)

Requirements related to dependability

Requirement	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired

Reliability versus availability

Reliability $R(t)$ of component C

Conditional probability that C has been functioning correctly during $[0, t)$ given C was functioning correctly at time $T = 0$.

Traditional metrics

- **Mean Time To Failure** ($MTTF$): The average time until a component fails.
- **Mean Time To Repair** ($MTTR$): The average time needed to repair a component.
- **Mean Time Between Failures** ($MTBF$): Simply $MTTF + MTTR$.

MTTF: tempo esperado até a primeira ocorrência de defeito

MTTR: tempo médio para reparo do sistema

MTBF: tempo médio entre os defeitos do sistema

Reliability versus availability

Availability $A(t)$ of component C

Average fraction of time that C has been up-and-running in interval $[0, t)$.

- Long-term availability A : $A(\infty)$
- **Note:** $A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF + MTTR}$

Observation

Reliability and availability make sense only if we have an accurate notion of what a failure actually is.

Terminology

Failure, error, fault

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

Failure = defeito

Fault = falha

Terminology

Handling faults

Term	Description	Example
Fault prevention	Prevent the occurrence of a fault	Don't hire sloppy programmers
Fault tolerance	Build a component such that it can mask the occurrence of a fault	Build each component by two independent programmers
Fault removal	Reduce the presence, number, or seriousness of a fault	Get rid of sloppy programmers
Fault forecasting	Estimate current presence, future incidence, and consequences of faults	Estimate how a recruiter is doing when it comes to hiring sloppy programmers

Failure models

Types of failures

Type	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	Fails to respond to incoming requests Fails to receive incoming messages Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure <i>Value failure</i> <i>State-transition failure</i>	Response is incorrect The value of the response is wrong Deviates from the correct flow of control
Arbitrary failure <i>Byzantine failure</i>	May produce arbitrary responses at arbitrary times

Dependability versus security

Omission versus commission

Arbitrary failures are sometimes qualified as **malicious**. It is better to make the following distinction:

- **Omission failures**: a component fails to take an action that it should have taken
- **Commission failure**: a component takes an action that it should not have taken

Dependability versus security

Omission versus commission

Arbitrary failures are sometimes qualified as **malicious**. It is better to make the following distinction:

- **Omission failures**: a component fails to take an action that it should have taken
- **Commission failure**: a component takes an action that it should not have taken

Observation

Note that **deliberate** failures, be they omission or commission failures are typically security problems. Distinguishing between deliberate failures and unintentional ones is, in general, impossible.

Halting failures

halt = parar/travar

Scenario

C no longer perceives any activity from C^* — a **halting failure**? Distinguishing between a **crash** or **omission/timing failure** may be impossible.

Asynchronous versus synchronous systems

- **Asynchronous system**: no assumptions about process execution speeds or message delivery times → **cannot reliably detect crash failures**.
- **Synchronous system**: process execution speeds and message delivery times are bounded → **we can reliably detect omission and timing failures**.
- In practice we have **partially synchronous systems**: most of the time, we can assume the system to be synchronous, yet there is no bound on the time that a system is asynchronous → **can normally reliably detect crash failures**.

Halting failures

Assumptions we can make

Halting type	Description
Fail-stop	Crash failures, but reliably detectable
Fail-noisy	Crash failures, eventually reliably detectable
Fail-silent	Omission or crash failures: clients cannot tell what went wrong
Fail-safe	Arbitrary, yet benign failures (i.e., they cannot do any harm)
Fail-arbitrary	Arbitrary, with malicious failures

Redundancy for failure masking

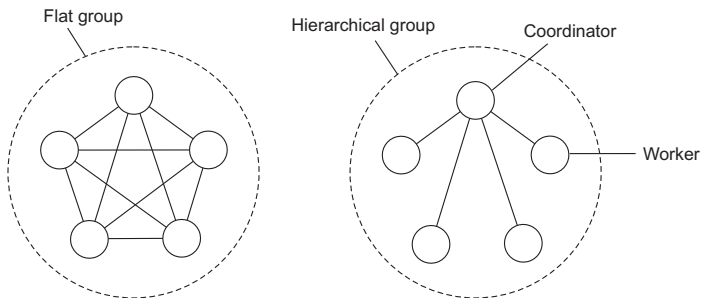
Types of redundancy

- **Information redundancy**: Add extra bits to data units so that errors can be recovered when bits are garbled.
- **Time redundancy**: Design a system such that an action can be performed again if anything went wrong. Typically used when faults are transient or intermittent.
- **Physical redundancy**: add equipment or processes in order to allow one or more components to fail. This type is extensively used in distributed systems.

Process resilience

Basic idea

Protect against malfunctioning processes through **process replication**, organizing multiple processes into **process group**. Distinguish between **flat groups** and **hierarchical groups**.



Groups and failure masking

k -fault tolerant group

When a group can mask any k concurrent member failures (k is called **degree of fault tolerance**).

Groups and failure masking

k -fault tolerant group

When a group can mask any k concurrent member failures (k is called **degree of fault tolerance**).

How large does a k -fault tolerant group need to be?

- With **halting failures** (crash/omission/timing failures): we need a total of $k + 1$ members as **no member will produce an incorrect result, so the result of one member is good enough**.
- With **arbitrary failures**: we need $2k + 1$ members so that the correct result can be obtained through a majority vote.

Groups and failure masking

k -fault tolerant group

When a group can mask any k concurrent member failures (k is called **degree of fault tolerance**).

How large does a k -fault tolerant group need to be?

- With **halting failures** (crash/omission/timing failures): we need a total of $k + 1$ members as **no member will produce an incorrect result, so the result of one member is good enough**.
- With **arbitrary failures**: we need $2k + 1$ members so that the correct result can be obtained through a majority vote.

Important assumptions

- All members are identical
- All members process commands in the same order

Result: We can now be sure that all processes do exactly the same thing.

Consensus

Prerequisite

In a fault-tolerant process group, each nonfaulty process executes the same commands, and in the same order, as every other nonfaulty process.

Reformulation

Nonfaulty group members need to reach **consensus** on which command to execute next.

Flooding-based consensus

System model

- A process group $\mathbf{P} = \{P_1, \dots, P_n\}$
- **Fail-stop** failure semantics, i.e., with **reliable failure detection**
- A client contacts a P_i requesting it to execute a command
- Every P_i maintains a list of proposed commands

Flooding-based consensus

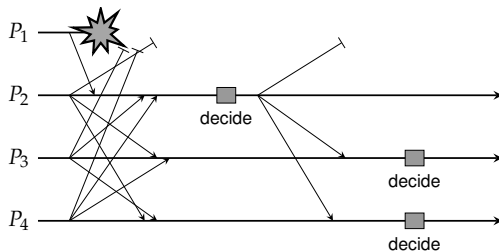
System model

- A process group $\mathbf{P} = \{P_1, \dots, P_n\}$
- **Fail-stop** failure semantics, i.e., with **reliable failure detection**
- A client contacts a P_i requesting it to execute a command
- Every P_i maintains a list of proposed commands

Basic algorithm (based on rounds)

- 1 In **round** r , P_i multicasts its known set of commands $\mathbf{C}_i^r$ to all others
- 2 At the end of r , each P_i merges all received commands into a new $\mathbf{C}_i^{r+1}$.
- 3 Next command cmd_i selected through a **globally shared, deterministic function**: $cmd_i \leftarrow select(\mathbf{C}_i^{r+1})$.

Flooding-based consensus: Example



Observations

- P_2 received all proposed commands from all other processes $\Rightarrow$ **makes decision**.
- P_3 may have detected that P_1 crashed, but does not know if P_2 received anything, i.e., P_3 cannot know **if it has the same information** as $P_2 \Rightarrow$ **cannot make decision** (same for P_4).

Realizing fault tolerance

Observation

Considering that the members in a fault-tolerant process group are so tightly coupled, we may bump into considerable performance problems, but perhaps even situations in which realizing fault tolerance is impossible.

Question

Are there limitations to what can be readily achieved?

- What is needed to enable reaching consensus?
- What happens when groups are partitioned?

Distributed consensus: when can it be reached

		Message ordering				
		Unordered		Ordered		
Process behavior	Synchronous	X	X	X	X	Bounded
				X	X	Unbounded
	Asynchronous				X	Bounded
					X	Unbounded
		Unicast	Multicast	Unicast	Multicast	
Message transmission						

Communication delay

Formal requirements for consensus

- Processes produce the same output value
- Every output value must be valid
- Every process must eventually provide output

Consistency, availability, and partitioning

CAP theorem

Any networked system providing shared data can provide only two of the following three properties:

- C:** **consistency**, by which a shared and replicated data item appears as a single, up-to-date copy
- A:** **availability**, by which updates will always be eventually executed
- P:** Tolerant to the **partitioning** of process group.

Conclusion

In a network subject to communication failures, it is impossible to realize an atomic read/write **shared memory** that guarantees a response to every request.

Failure detection

Issue

How can we **reliably detect** that a process has **actually crashed**?

General model

- Each process is equipped with a failure detection module
- A process P **probes** another process Q for a reaction
- If Q reacts: Q is considered to be alive (by P)
- If Q does not react with t time units: Q is **suspected** to have crashed

Observation for a **synchronous** system

a suspected crash $\equiv$ a known crash

Practical failure detection

Implementation

- If P did not receive **heartbeat** from Q within time t : P suspects Q .
- If Q later sends a message (which is received by P):
 - P stops suspecting Q
 - P increases the timeout value t
- **Note:** if Q did crash, P will keep suspecting Q .

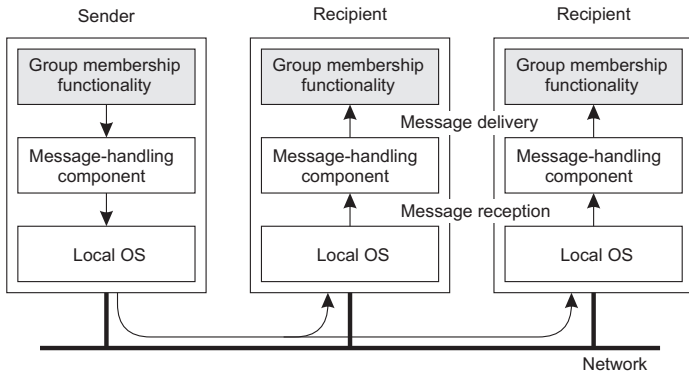
Point-to-point communication

- reliable point-to-point communication is established using a reliable transport protocol, such as TCP.
- TCP masks omission failures by using ACKs and retransmissions.
- Crash failures are not masked.

Simple reliable group communication

Intuition

A message sent to a process group **G** should be delivered to each member of **G**. **Important**: make distinction between receiving and delivering messages.



Less simple reliable group communication

Reliable communication in the presence of faulty processes

Group communication is reliable when it can be guaranteed that a message is **received and subsequently delivered** by all **nonfaulty group members**.

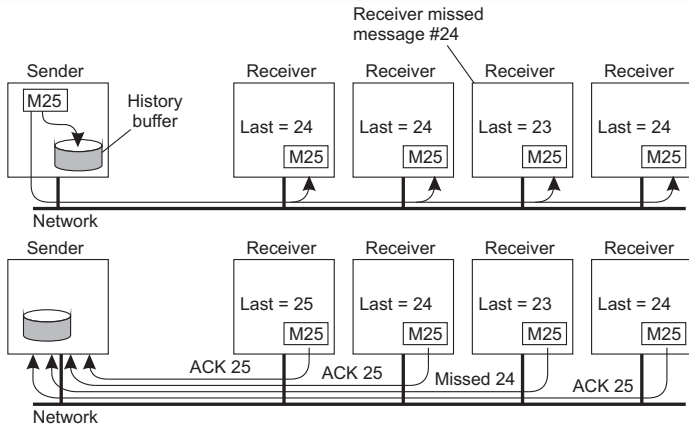
Tricky part

Agreement is needed on what the group actually looks like before a received message can be delivered.

Simple reliable group communication

Reliable communication, but assume nonfaulty processes

Reliable group communication now boils down to **reliable multicasting**: is a message received and delivered to each recipient, **as intended by the sender**.



Distributed commit protocols

Problem

Have an operation being performed by each member of a process group, or none at all.

- **Reliable multicasting**: a message is to be delivered to all recipients.
- **Distributed transaction**: each local transaction must succeed.

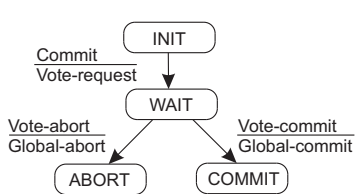
Two-phase commit protocol (2PC)

Essence

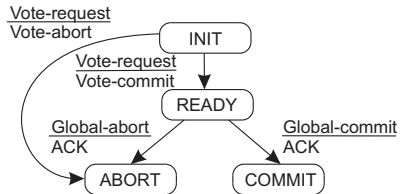
The client who initiated the computation acts as **coordinator**; processes required to commit are the **participants**.

- **Phase 1a:** Coordinator sends VOTE-REQUEST to participants (also called a **pre-write**)
- **Phase 1b:** When participant receives VOTE-REQUEST it returns either VOTE-COMMIT or VOTE-ABORT to coordinator. If it sends VOTE-ABORT, it aborts its local computation
- **Phase 2a:** Coordinator collects all votes; if all are VOTE-COMMIT, it sends GLOBAL-COMMIT to all participants, otherwise it sends GLOBAL-ABORT
- **Phase 2b:** Each participant waits for GLOBAL-COMMIT or GLOBAL-ABORT and handles accordingly.

2PC - Finite state machines



Coordinator



Participant

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- *INIT*: decide to abort and informs the coordinator

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- **READY**: Participant is waiting to either commit or abort. After recovery, participant needs to know which state transition it should make $\Rightarrow$ contact other process

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- **ABORT**: Merely make entry into abort state **idempotent**, e.g., removing the workspace of results

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- **COMMIT**: Also make entry into commit state idempotent, e.g., copying workspace to storage.

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- **INIT**: decide to abort and informs the coordinator
- **READY**: Participant is waiting to either commit or abort. After recovery, participant needs to know which state transition it should make $\Rightarrow$ contact other process
- **ABORT**: Merely make entry into abort state **idempotent**, e.g., removing the workspace of results
- **COMMIT**: Also make entry into commit state idempotent, e.g., copying workspace to storage.

Observation

When distributed commit is required, having participants use temporary workspaces to keep their results allows for simple recovery in the presence of failures.

2PC – Failing participant

W

hen a recovery is needed to *READY* state, check state of other participants.

Recovering participant *P* contacts another participant *Q*

State of <i>Q</i>	Action by <i>P</i>
<i>COMMIT</i>	Make transition to <i>COMMIT</i>
<i>ABORT</i>	Make transition to <i>ABORT</i>
<i>INIT</i>	Make transition to <i>ABORT</i>
<i>READY</i>	Contact another participant

Result

If all participants are in the *READY* state, the protocol blocks. Apparently, the coordinator is failing. **Note:** The protocol prescribes that we need the decision from the coordinator.

2PC – Failing coordinator

Observation

The real problem lies in the fact that the coordinator's final decision may not be available for some time (or actually lost).

Alternative

Let a participant P in the *READY* state timeout when it hasn't received the coordinator's decision; P tries to find out what other participants know (as discussed).

Observation

Essence of the problem is that a recovering participant cannot make a **local** decision: it is dependent on other (possibly failed) processes

Recovery: Background

Essence

When a failure occurs, we need to bring the system into an error-free state:

- **Forward error recovery**: Find a new state from which the system can continue operation
- **Backward error recovery**: Bring the system back into a **previous** error-free state

Practice

Use backward error recovery, requiring that we establish **recovery points**

Observation

Recovery in distributed systems is complicated by the fact that processes need to cooperate in identifying a **consistent state** from where to recover

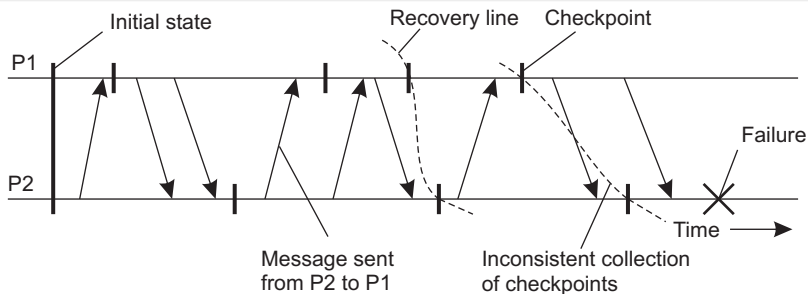
Consistent recovery state

Requirement

Every message that has been received is also shown to have been sent in the state of the sender.

Recovery line

Assuming processes regularly **checkpoint** their state, the most recent **consistent global checkpoint**.



Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a **checkpoint request** message

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a **checkpoint request** message
- When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a **checkpoint request** message
- When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- When all checkpoints have been confirmed at the coordinator, the latter broadcasts a **checkpoint done** message to allow all processes to continue

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a **checkpoint request** message
- When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- When all checkpoints have been confirmed at the coordinator, the latter broadcasts a **checkpoint done** message to allow all processes to continue

Observation

It is possible to consider only those processes that depend on the recovery of the coordinator, and ignore the rest

Message logging

Alternative

Instead of taking an (expensive) checkpoint, try to **replay** your (communication) behavior from the most recent checkpoint $\Rightarrow$ store messages in a log.

Assumption

We assume a **piecewise deterministic** execution model:

- The execution of each process can be considered as a sequence of state intervals
- Each state interval starts with a nondeterministic event (e.g., message receipt)
- Execution in a state interval is deterministic

Conclusion

If we record nondeterministic events (to replay them later), we obtain a deterministic execution model that will allow us to do a complete replay.